CONSULTOS

Production-Level Frontend Generation Prompt

A complete implementation specification for Google AI Studio, aligned with the ConsultOS FastAPI backend architecture and designed for subsequent integration through Antigravity CLI.

Goal — Generate a coherent, implementation-ready React frontend for an autonomous business investigation and workflow platform. The frontend must display real backend state, not simulate functionality with hardcoded results.

Frontend baseline	Integration baseline	Primary demo
React + Vite + JavaScript + Tailwind CSS	FastAPI REST API + SQLite persistence + SSE execution stream	Investigate why business revenue declined, show evidence, replan from new evidence, and generate a verified report

Use this document as the single source of truth when generating the frontend in Google AI Studio. Import the resulting project into the existing ConsultOS repository and reconcile it with the backend before running it.

Important: the API routes below are the agreed frontend integration contract from the current architecture plan. If the actual backend differs, inspect its code and adapt the frontend to the implemented contract rather than silently inventing endpoints.

1. MASTER PROMPT — ROLE AND MISSION

Copy the following specification into Google AI Studio. Treat it as an implementation task, not a design-only request.

ROLE
Act as a principal frontend engineer, enterprise product designer, and frontend integration architect. Build the complete ConsultOS frontend as a maintainable, responsive, accessible React application. You are responsible for producing real source files and working interactions, not merely a mockup, screenshot, or high-level plan.

PRODUCT
ConsultOS is an Autonomous Business Investigation & Workflow Platform. It transforms a business question into a structured investigation plan, executes approved analytical tasks with dependencies, records evidence, revises hypotheses when new evidence changes the picture, recovers from eligible task failures, and produces an evidence-backed report with visible execution history.

PRIMARY OUTCOME
Deliver a polished frontend that connects to a FastAPI backend and supports the full user journey: create investigation → upload CSV → run analysis → observe tasks → inspect evidence → see replanning and recovery → review final report and metrics.

Non-negotiable rules

- Use React, Vite, JavaScript (not TypeScript), and Tailwind CSS unless the existing repository already has a compatible, working alternative.
- Do not invent API responses, investigation results, task events, metrics, or report findings. Render only backend data or clearly labelled local design placeholders.
- Do not replace the backend with browser-only business logic. Analytical calculations, task execution, dependency enforcement, evidence creation, and replanning belong to the backend.
- Do not use a single giant component. Use modular pages, shared components, hooks, and a centralized API client.
- Do not leave primary buttons decorative. Every visible action must work, be disabled when invalid, or clearly communicate that it is not yet supported.
- Do not silently change the backend contract. Inspect the actual API implementation and align to it.
- Do not stop after giving a plan. Generate the complete source code, configuration, styles, and concise setup instructions.

2. DESIGN SYSTEM AND VISUAL DIRECTION

Design concept

Design ConsultOS as a high-trust investigative intelligence workspace: precise, calm, information-dense without clutter, and credible to business analysts and technical evaluators. The interface should feel like a professional analytical operations product, not a generic AI chat application.

Visual system

Area	Requirement
Theme	Dark-first dashboard with a near-black/navy canvas, elevated panels, restrained violet accent, muted slate text, and semantic status colors. Ensure contrast and legibility.
Typography	Use a clean UI sans-serif. Clear hierarchy for page titles, section headings, metadata, metrics, tables, and long-form evidence.
Layout	Desktop-first for laptop judging, responsive at tablet and mobile widths. Use consistent spacing, restrained borders, clear panel grouping, and a stable navigation shell.
Status	Use text labels plus icons/shape—not color alone—for pending, ready, running, completed, failed, blocked, and skipped.
Motion	Subtle transitions only. Respect reduced-motion preferences. Never use animation to imply work that the backend has not performed.
Data display	Format timestamps, percentages, durations, counts, and numerical results consistently. Show units and comparison periods where available.
Empty/error states	Design useful empty, loading, retry, permission/error, and partial-result states. Never leave blank panels or endless spinners.

Core shell

- Left sidebar on desktop with ConsultOS mark, Dashboard, Investigations, and Evaluation/Activity navigation if those backend capabilities exist.
- Top header with current page, backend connectivity indicator, and relevant context.
- Main workspace with clear title, primary action, and content sections.
- Mobile navigation that does not obstruct the primary workflow.
- A visible but non-alarming connection state: Connected, Reconnecting, or Offline.

Avoid excessive gradients, glowing borders, decorative AI imagery, oversized hero sections, fake activity feeds, and large empty cards. Information architecture and live workflow clarity take priority.

3. PAGE-BY-PAGE SPECIFICATION

A. Dashboard

- Show total, running, completed, and failed investigations using backend-derived counts.
- Show recent investigations with question/title, created time, current status, task progress if available, and an Open action.
- Provide a clear New Investigation CTA.
- Show recent real execution events only if the API exposes them.
- Include loading, empty, partial-data, and API-failure states.
- Never hardcode dashboard metrics as if they were live.

B. Create Investigation

- Provide a large, labelled business-question input with helpful example text, but do not pre-submit an example.
- Provide CSV upload with filename, file size, file type, upload progress if available, remove/replace action, and validation errors.
- Explain accepted file types and limits using actual backend constraints. Do not promise PDF/DOCX/XLSX support unless implemented.
- Validate empty questions and missing required files when required by the backend.
- Submit through the API, prevent accidental duplicate submissions, and route to the newly created investigation after success.

- Show clear error messages for unsupported files, malformed data, network failures, and server validation errors.

C. Investigation Workspace

- Show the business question, investigation ID, creation/update times, status, current plan version, and primary actions.
- Show the dependency graph and a complementary task list/table for accessibility and small screens.
- Display each task's title, type, status, dependencies, attempts, duration, and failure detail when available.
- Show current hypotheses and their statuses, with links to supporting or contradicting evidence.
- Show a live event/timeline feed driven by persisted backend events and SSE.
- Expose Retry only for eligible failed tasks; expose Cancel only while cancellation is valid.
- Show plan revisions distinctly and identify tasks added by replanning.
- Support partial failure: completed work must remain visible when another task fails.

D. Task Detail

- Display task description, status, dependency prerequisites, attempts, timestamps, duration, structured result, produced evidence, and error details.
- Clearly show why a task is pending or blocked, if the backend provides that reason.
- Show retry controls only when permitted by the backend.
- Do not fabricate task logs or tool outputs.

E. Evidence Explorer

- List evidence records with evidence ID, source filename/identifier, producing task, relevant columns, observed result, calculation/transformation, units or period, and timestamp where present.
- Allow filtering by source, task, evidence type, and hypothesis when the available data supports it.
- Show evidence detail in a side panel or detail view.
- Link each report claim or hypothesis to the evidence IDs that support or contradict it.
- Represent missing evidence honestly; do not create fake citations.

F. Final Report

- Display business question, executive summary, methodology, findings, calculations, recommendations, assumptions, contradictory evidence, limitations, unresolved questions, and execution summary when available.
- Make evidence references clickable and navigate to the matching evidence detail.
- Distinguish observed facts from interpretation and recommendations.
- Show incomplete/partial status when tasks failed or evidence remains unresolved.
- Provide Download Markdown or HTML only if export is implemented correctly. Never create an export link that does nothing.
- Offer a clear 'Report not ready' state until the backend has produced a report.

G. Activity / Evaluation

If evaluation endpoints are implemented, provide a page showing test cases, evaluation run status, and available metrics. If the backend does not yet expose evaluation data, do not create fake charts. Keep the route out of primary navigation or label it as unavailable until wired to real data.

4. BACKEND INTEGRATION CONTRACT

Create one centralized API client, one configuration source for the API base URL, typed-by-convention JSDoc shapes or equivalent JavaScript documentation, and consistent error handling. The default local API base may be http://localhost:8000, configurable with Vite environment variables.

Suggested environment variable: VITE_API_BASE_URL=http://localhost:8000

Method	Endpoint	Frontend use
GET	/api/health	Backend connectivity check.
GET	/api/investigations	List investigations for dashboard and history.
POST	/api/investigations	Create an investigation; multipart upload if supported by actual backend.
GET	/api/investigations/{id}	Fetch investigation detail and current status.
POST	/api/investigations/{id}/run	Start execution.
GET	/api/investigations/{id}/tasks	Load task graph/list.
GET	/api/investigations/{id}/evidence	Load evidence records.
GET	/api/investigations/{id}/events	Load persisted event history.
GET	/api/investigations/{id}/report	Load final report; handle not-ready state.
POST	/api/investigations/{id}/retry/{task_id}	Request an eligible task retry.
POST	/api/investigations/{id}/cancel	Request cancellation when valid.
GET	/api/investigations/{id}/events/stream	Subscribe to SSE updates if implemented.

Important: these routes reflect the planned backend contract, not proof that every endpoint exists in the current code. Inspect backend routes before implementing calls. If the real route, request body, response shape, or upload method differs, use the actual contract and report the discrepancy.

API client behavior

- Centralize fetch calls in src/services/api.js (or equivalent).
- Use AbortController for cancellable requests and timeouts where appropriate.
- Parse JSON defensively; handle non-JSON error responses.
- Normalize errors into a message plus optional status/code/details.
- Do not retry non-idempotent POST requests automatically unless the backend supports idempotency.
- Avoid putting API secrets in frontend environment variables; browser-exposed VITE_ variables are public.
- Configure CORS on the backend rather than attempting to bypass it in the browser.

5. LIVE EXECUTION AND STATE MANAGEMENT

Server-Sent Events

Use native EventSource for the investigation stream if the backend actually implements SSE. Handle open, message, error, and cleanup. Close the stream when leaving the investigation or unmounting the page. Reconnect safely and re-fetch persisted state after reconnecting.

- Treat SSE as an update channel, not the source of truth. The persisted REST state remains authoritative.
- Deduplicate events using event IDs when available.
- Handle unknown event types without crashing.
- Show a visible reconnecting indicator if the stream disconnects.
- If SSE is unavailable, use bounded polling of the investigation and task endpoints; do not simulate a live stream.

State principles

- Backend state is authoritative for task/investigation status.

- Keep temporary form/upload state local to the relevant page.
- Avoid duplicated sources of truth for the same investigation.
- Refresh or invalidate relevant data after create, run, retry, or cancel actions.
- Do not show a task as completed before the backend confirms it.
- Support page refresh and direct URL navigation to an investigation.

6. COMPONENT AND FILE ORGANIZATION

Use a modular, understandable structure. Adapt to existing code rather than creating a second frontend root.

```
frontend/
  src/
    app/
      App.jsx
      router.jsx
    layouts/
      AppShell.jsx
      Sidebar.jsx
      TopBar.jsx
    pages/
      DashboardPage.jsx
      NewInvestigationPage.jsx
      InvestigationPage.jsx
      EvidencePage.jsx
      ReportPage.jsx
      NotFoundPage.jsx
    components/
      StatusBadge.jsx
      MetricCard.jsx
      EmptyState.jsx
      ErrorState.jsx
      LoadingState.jsx
      ConfirmDialog.jsx
      TaskGraph.jsx
      TaskList.jsx
      HypothesisPanel.jsx
      EvidenceCard.jsx
      EventTimeline.jsx
      ReportSection.jsx
      FileDropzone.jsx
    hooks/
      useInvestigation.js
      useInvestigationStream.js
    services/
      api.js
      investigations.js
    utils/
      formatters.js
      status.js
    styles/
      index.css
  .env.example
  package.json
```

Avoid unnecessary state libraries or graph libraries. If a dependency is added, justify its use, install it correctly, and ensure the project still builds.

7. DATA SHAPES AND RENDERING RULES

Use these as conceptual frontend shapes. Reconcile them with actual Pydantic response models and API output. Do not force the backend to match these examples without checking its current contract.

```
Investigation:
{
  id, business_question, status,
  current_plan_version, created_at, updated_at
```

```
}

Task:
{
  id, investigation_id, title, description, task_type,
  status, dependencies, attempts, result, error,
  created_at, updated_at
}

Evidence:
{
  id, investigation_id, task_id, source_type,
  source_reference, finding, supporting_data, created_at
}

Hypothesis:
{
  id, investigation_id, statement, status,
  supporting_evidence_ids, contradicting_evidence_ids,
  confidence, created_at
}

Event:
{
  id, investigation_id, task_id,
  event_type, details, created_at
}

Report:
{
  id, investigation_id, executive_summary,
  findings, recommendations, limitations,
  unresolved_questions, created_at
}
```

The actual backend may nest these objects or use different names. Build small adapter/normalization functions if needed; do not scatter assumptions about response shape throughout the UI.

Status presentation

State	UI treatment
draft / queued / pending	Neutral badge; explain that execution has not begun or is waiting.
running / ready	Active badge with clear text; avoid fake percentage progress unless backend provides it.
replanning	Distinct active state with current plan version and a timeline entry.
completed	Success badge only after backend confirmation.
failed	Error badge with available error summary and valid recovery action.
blocked	Explain dependency blockage if known; do not label as a tool failure.
cancelled / skipped	Neutral or warning badge with the correct meaning.

8. ACCESSIBILITY, RESPONSIVENESS AND QUALITY

- Use semantic HTML, proper headings, labelled form controls, accessible dialog behavior, and keyboard navigation.
- Maintain readable contrast and visible focus states.
- Do not use color as the sole status signal.
- Use responsive layouts: desktop multi-column views should collapse cleanly on narrow screens.
- Provide skeletons or restrained loading states without shifting the entire layout unnecessarily.
- Show actionable validation and error messages near the relevant control.

- Avoid horizontal overflow in tables; use responsive cards or controlled horizontal scrolling.
- Respect prefers-reduced-motion.
- Use stable keys, avoid stale asynchronous updates, and clean up subscriptions and timers.

9. SECURITY AND TRUST

- Do not expose API keys, provider credentials, database credentials, or server secrets in frontend code.
- Treat business questions, filenames, evidence text, and report content as untrusted text. Render them as text, not raw HTML.
- Do not use dangerouslySetInnerHTML for backend-supplied content unless it is safely sanitized by a deliberate, tested policy.
- Do not expose local filesystem paths or internal stack traces to end users.
- Do not assume a hidden UI control is authorization. The backend must enforce valid state transitions and permissions.
- Use confirmation for consequential actions such as cancelling an active investigation.
- Clearly label incomplete findings and unsupported formats.
- Do not claim that the application is secure or production-ready solely because frontend linting passes.

10. ERROR, LOADING AND EMPTY-STATE MATRIX

Scenario	Required behavior
Backend offline	Show Offline/Disconnected state, explain API unavailability, offer a retry action, and avoid endless spinners.
No investigations	Explain the empty state and provide New Investigation CTA.
Invalid CSV	Show backend validation message and retain the question/form where safe.
Investigation still running	Show current status, known tasks/events, and a report-not-ready state.
Task failed	Show task error summary, preserve successful results, and display Retry only if allowed.
SSE disconnected	Show reconnecting state, fetch latest REST state, and reconnect safely.
Partial results	Show completed findings and clearly label the investigation as partial or failed.
Report unavailable	Explain that the report is not ready or the API returned an error; do not create a fake report.
Unknown investigation ID	Show a not-found page/state with a route back to investigations.

11. IMPLEMENTATION ORDER

Implement in this order to reduce integration risk:

- Audit existing frontend files and the actual backend API routes/models.
- Establish Vite entry point, Tailwind configuration, routing, and app shell.
- Build the centralized API client and verify health connectivity.
- Build dashboard and investigation creation against real endpoints.
- Build investigation workspace, task list/graph, and status presentation.
- Add evidence explorer and report views.
- Add SSE or bounded polling for live execution.
- Add retry/cancel actions with backend state validation.
- Add responsive, accessible loading/error/empty states.

- Run lint/build and complete a real integration journey.

12. ACCEPTANCE CRITERIA

- The frontend installs and builds successfully in the target repository.
- The app can detect whether the FastAPI backend is reachable.
- A user can create an investigation and open its detail page.
- CSV upload uses the backend's actual supported request format.
- The workspace displays real tasks, dependencies, hypotheses, evidence, and events when provided by the API.
- Task status updates come from backend responses or real SSE events.
- Replanning is visible only when a real backend plan revision/event exists.
- The final report renders actual backend findings and evidence references.
- Retry and cancel actions call real endpoints and handle invalid states.
- Refresh/direct navigation preserves access to persisted investigations.
- No primary control is decorative or backed by fake success behavior.
- The interface works at laptop and mobile widths.
- No console errors or broken imports remain in the main journey.
- The production build succeeds and its result is reported honestly.

13. FINAL OUTPUT REQUIRED FROM GOOGLE AI STUDIO

Generate the complete frontend project files. If the environment cannot emit the entire project in one response, generate the project in coherent file groups while preserving the same architecture and API contracts. Do not omit essential files and do not replace implementations with TODO comments.

- Provide the complete React/Vite/JavaScript source code and Tailwind configuration.
- Provide package.json and all required dependency declarations.
- Provide .env.example with the frontend API base variable.
- Provide a README with exact install/start/build commands.
- Provide an API integration map documenting each endpoint actually used.
- List any backend route or schema mismatches discovered.
- List any feature that remains unimplemented or uses a clearly labelled fallback.
- Do not claim that a build or integration test passed unless it was actually executed.

Final instruction: build the best complete frontend possible while preserving the backend contract. Favor a coherent, functioning end-to-end product over superficial visual complexity. Do not stop at a design proposal.

14. ANTIGRAVITY INTEGRATION CHECKLIST

- Import the generated frontend into the existing ConsultOS repository; do not overwrite the backend directory.
- Inspect the generated package.json and install only the required dependencies.
- Compare frontend API calls against the real FastAPI routes and Pydantic schemas.
- Resolve any differences in upload format, IDs, status names, and response nesting.
- Set VITE_API_BASE_URL in the frontend environment file.
- Confirm backend CORS allows the local frontend origin.
- Run the frontend and backend in separate terminals.

- Test create → run → task progress → evidence → report using a real dataset.
- Refresh the investigation page and confirm state is fetched from persistence.
- Run the production build and fix integration errors before adding cosmetic features.

Suggested local startup (adjust paths and commands to the actual repository):

```
# Terminal 1 — backend
cd backend
# activate the project's existing virtual environment
uvicorn app.main:app --reload --port 8000

# Terminal 2 — frontend
cd frontend
npm install
npm run dev
```

Do not run these commands blindly if the actual entry point or package scripts differ. Inspect the repository first.